Data Analytics Project:Car Data Dataset Analysis

Dataset Used: cardata.csv

Team Members: Swetha| Sumanth | Sruthi | Subbaraju | Tarun

This document outlines a structured, end-to-end data analysis project based on the Car Data dataset. The project is divided into ten scenarios covering data cleaning, visualization, and advanced analysis.

Project Goal: To analyse car selling prices, fuel type distributions, and pricing patterns using the cardata.csv dataset.

The primary libraries/modules required for this project are:

- Pandas: For all data manipulation, filtering, aggregation, and feature engineering tasks.
- NumPy: Specifically for numerical calculations like price differences (np.diff).
- Visualization Library (e.g., Matplotlib): For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Scatter Plots, Histograms).

Project Coverage:

Category	Skills Covered
Data Pre-processing	Data Loading, Missing Value Handling (Mean/Mode), Type Conversion
Pandas Operations	Grouping, Filtering, Aggregation, Feature Engineering (Car Age Category)
NumPy Calculations	Array conversion, Price difference calculation (np.diff)
Visualization	Line Chart, Bar Chart, Pie Chart, Scatter Plot, Histogram, Saving Plots

Scenario 1: Data Loading & Basic Cleaning

Description:

In this scenario we are loading the dataset to the pandas DataFrame and displaying the first 5 rows, last 5 rows, and column names. After that we are checking for any missing values in Selling_Price, Present_Price, Kms_Driven, and Fuel_Type columns. If found, we replace numeric columns with mean and Fuel_Type with mode. We also convert numeric columns to proper numeric type and use NumPy for basic statistical calculations.

Task	Description
1.	Load the dataset using Pandas.
2.	Display: First 5 rows, Last 5 rows, Column names, Shape of dataset.
3.	Check data types of all columns.
4.	Check for missing values in Selling_Price, Present_Price, Kms_Driven, Fuel_Type.
5.	Fill missing values: Selling_Price, Present_Price, Kms_Driven with mean; Fuel_Type with mode.
6.	Convert Selling_Price, Present_Price, Kms_Driven, Year to numeric type if required.
7.	Convert Selling_Price and Kms_Driven into NumPy arrays.
8.	Use NumPy to calculate minimum, maximum, and average selling price.

Output:

Image 1.1

```
  Car_Name  Year  Selling_Price  Present_Price  Kms_Driven Fuel_Type Seller_Type Transmission  Owner
0     ritz  2014           3.35           5.59       27000    Petrol      Dealer       Manual      0
1      sx4  2013           4.75           9.54       43000    Diesel      Dealer       Manual      0
2     ciaz  2017           7.25           9.85        6900    Petrol      Dealer       Manual      0
3  wagon r  2011           2.85           4.15        5200    Petrol      Dealer       Manual      0
4    swift  2014           4.60           6.87       42450    Diesel      Dealer       Manual      0
    Car_Name  Year  Selling_Price  Present_Price  Kms_Driven Fuel_Type Seller_Type Transmission  Owner
296     city  2016           9.50           11.6       33988    Diesel      Dealer       Manual      0
297     brio  2015           4.00            5.9       60000    Petrol      Dealer       Manual      0
298     city  2009           3.35           11.0       87934    Petrol      Dealer       Manual      0
299     city  2017          11.50           12.5        9000    Diesel      Dealer       Manual      0
300     brio  2016           5.30            5.9        5464    Petrol      Dealer       Manual      0
```

Image 1.2

```
Index(['Car_Name', 'Year', 'Selling_Price', 'Present_Price', 'Kms_Driven',
       'Fuel_Type', 'Seller_Type', 'Transmission', 'Owner'],
      dtype='str')
(301, 9)
Car_Name             str
Year               int64
Selling_Price    float64
Present_Price    float64
Kms_Driven         int64
Fuel_Type            str
Seller_Type          str
Transmission         str
Owner              int64
dtype: object
Selling_Price    0
Present_Price    0
Kms_Driven       0
Fuel_Type        0
dtype: int64
```

Conclusion:

In this scenario we loaded the CSV file to a pandas DataFrame and displayed the first and last 5 rows with column names and shape. We checked for missing values in key columns and handled them appropriately using mean/mode. The numeric columns were confirmed or converted to their proper types. Using NumPy arrays, we computed the minimum, maximum, and average selling prices for statistical insight.

Scenario 2: Selling Price Trend (Line Graph)

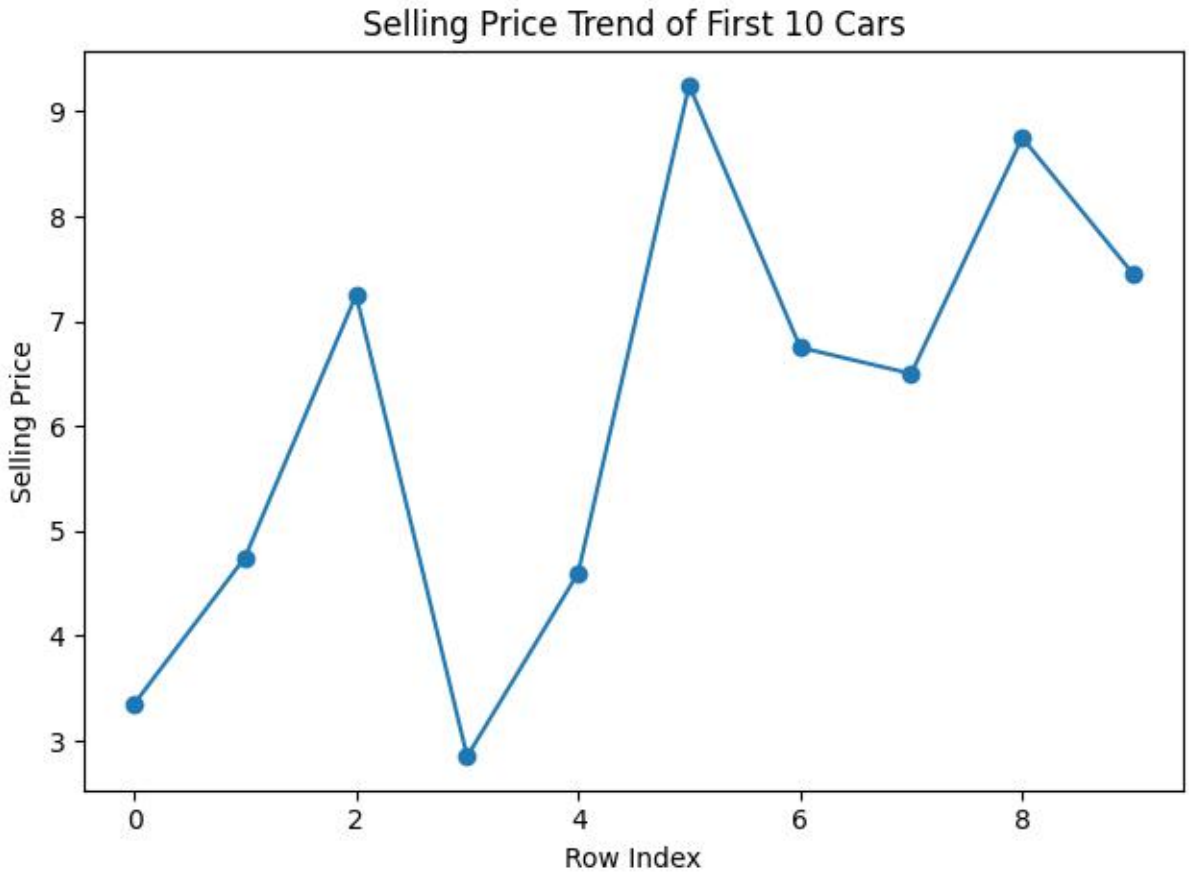
Description:

In this scenario, we work with car data to analyze selling price trends. Using the top 10 values from columns Car_Name and Selling_Price. The Selling_Price values are then converted into a NumPy array. We are using line graph to represent selling prices.

Task	Description
1.	Select Car_Name and Selling_Price columns.
2.	Take the first 10 rows only using Pandas.
3.	Convert Selling_Price into a NumPy array.
4.	Plot a line graph: X-axis → row index (0-9), Y-axis → Selling Price.
5.	Add title, x-axis label, y-axis label, and markers.
6.	Save the graph with a suitable filename.

Graph:

Here in the Line graph, X-axis represents the index values (0-9) and the Y-axis represents the selling price of top 10 cars as shown below.



Conclusion:

In this scenario we used the top 10 values from selling_price column and converted the data into numpy array. After that the line graph is created to show easily understand the changes in prices across different cars. And the graph is saved for future references.

Scenario 3: Expensive Cars Analysis (Filtering + Bar Chart)

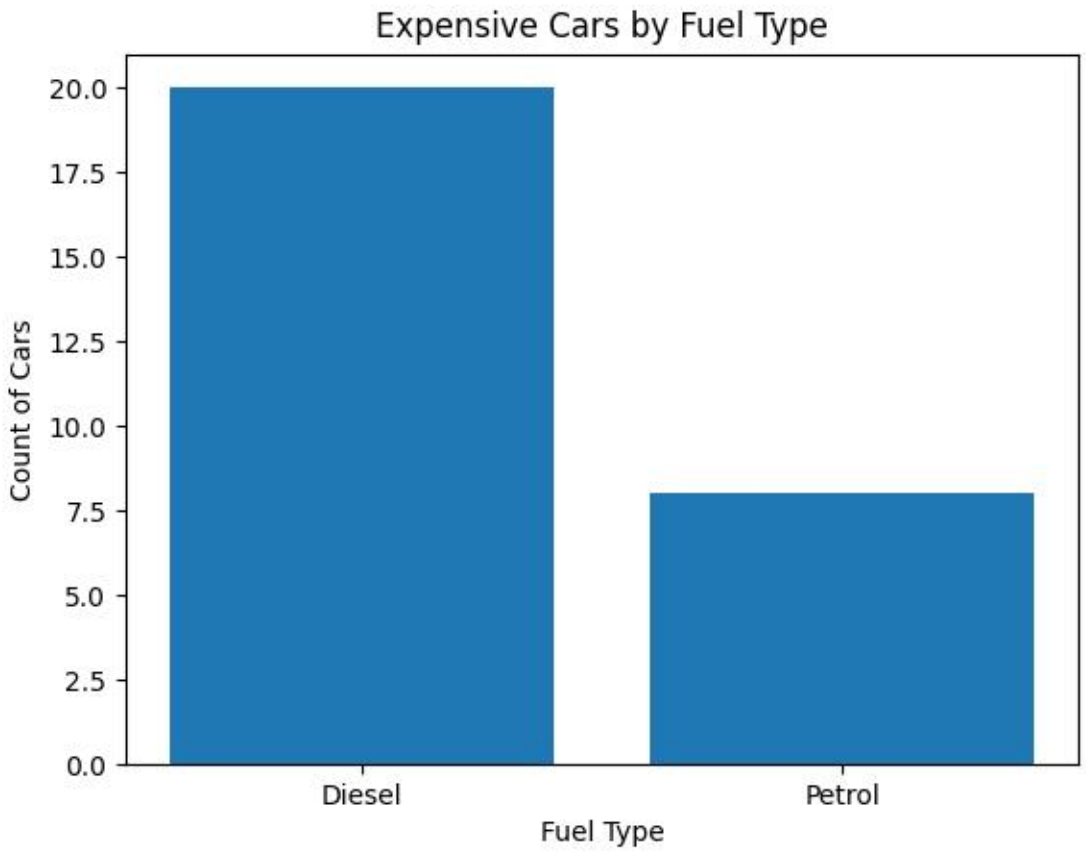
Description:

In this scenario, we analyze expensive cars by focusing on those with a selling price greater than 10. The filtered data is then grouped based on fuel type (such as Petrol or Diesel), and the number of cars in each category is counted. These results are converted into NumPy arrays to prepare them for visualization.A bar chart is created where the X-axis represents fuel types and the Y-axis represents the count of expensive cars. Titles and labels are added to improve clarity, and the graph is saved as an image file for future use.

Task	Description
1.	Filter cars where Selling_Price > 10.
2.	Group the filtered data by Fuel_Type.
3.	Count the number of cars in each fuel type.
4.	Convert fuel type labels and counts into NumPy arrays.
5.	Plot a bar chart: X-axis → Fuel Type, Y-axis → Count of expensive cars.
6.	Add title, x-label, and y-label.
7.	Save the graph.

Graph:

The graph is a bar chart where the X-axis represents the fuel type and the Y-axis represents the count of expensive cars (Selling_Price > 10). Each bar shows how many high-priced cars belong to a specific fuel type category. Taller bars indicate more expensive cars in that fuel type.



Conclusion:

This analysis identifies which fuel types are most common among expensive cars. By filtering cars with higher selling prices and counting them by fuel type, we can clearly see the dominant fuel category. The bar chart helps in easy comparison and visualization of distribution.

Scenario 4: Fuel Type Distribution (Pie Chart)

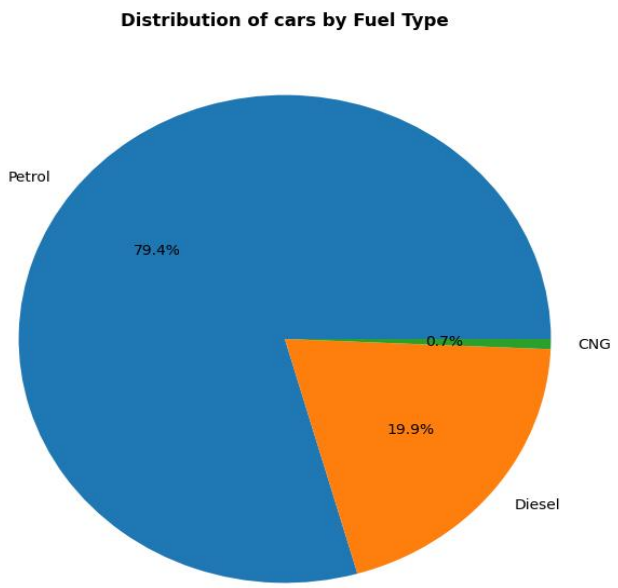
Description:

This analysis examines how cars are distributed based on their fuel type. It helps identify the most commonly used fuel among vehicles in the dataset. The data is grouped by fuel category and counted. A pie chart is used to visually represent the proportion of each fuel type.

Task	Description
1.	Count the number of cars in each Fuel_Type.
2.	Select all categories or top categories if needed.
3.	Prepare labels and values.
4.	Convert values into a NumPy array.
5.	Plot a pie chart using Matplotlib.
6.	Add percentage labels (autopct) and title.
7.	Save the graph.

Graph:

The pie chart displays different fuel types as slices, with each slice representing its percentage share. Labels indicate fuel categories such as Petrol, Diesel, and others. Percentage values are shown on each slice for clarity. The chart title highlights the purpose of fuel type distribution.



Conclusion:

The dominant fuel type can be clearly identified from the largest slice. This insight helps understand market preference and trends in fuel usage. Lesser-used fuel types appear as smaller slices. Overall, the chart provides a quick comparison of fuel popularity.

Scenario 5: Present Price vs Selling Price (Scatter Plot)

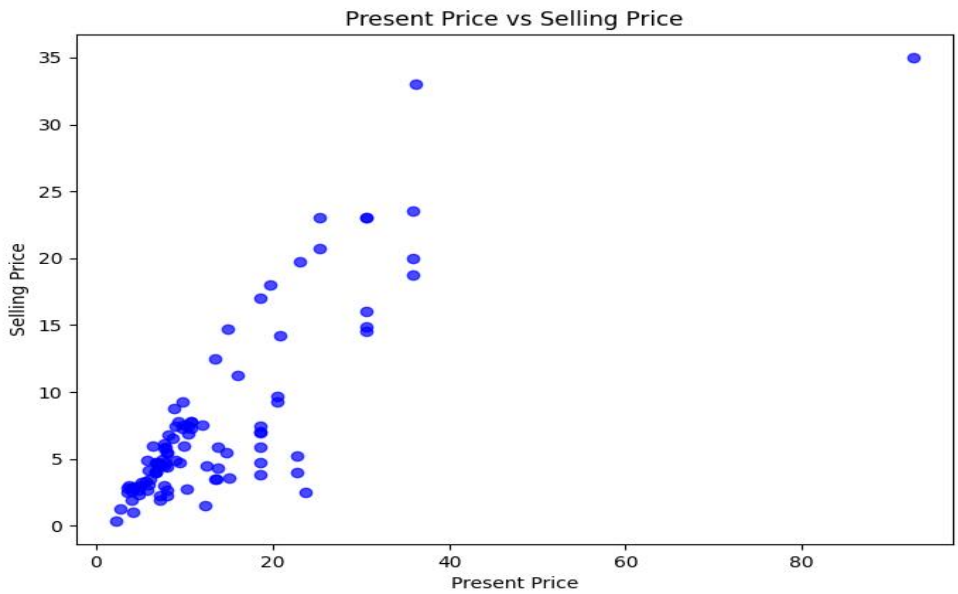
Description:

This visualization compares a car's current market value with its selling price using a scatter plot. After cleaning and sampling the data, both variables are plotted to observe their relationship. The plot typically shows a **positive correlation**, meaning cars with higher present prices tend to have higher selling prices. This indicates that market value strongly influences resale price.

Task	Description
1.	Select Present_Price and Selling_Price columns.
2.	Remove missing values if any.
3.	Take a smaller sample (first 50 or 100 rows) using Pandas.
4.	Convert both columns into NumPy arrays.
5.	Plot a scatter plot: X-axis → Present_Price, Y-axis → Selling_Price.
6.	Add title, x-label, and y-label.
7.	Observe whether there is a positive relationship.
8.	Save the graph.

Graph:

The scatter plot shows the relationship between **Present_Price** on the X-axis and **Selling_Price** on the Y-axis. Each dot represents a car in the dataset. The points generally move upward from left to right, indicating a positive trend. This means cars with higher present prices usually have higher selling prices, though slightly lower due to depreciation.



Conclusion:

The scatter plot shows a strong positive relationship between present price and selling price. As the present price increases, the selling price also increases in most cases. However, most points lie below the equal-value line, indicating depreciation. The spread of points suggests that other factors also influence selling price. Overall, present price is a key predictor of selling price.

Scenario 6: Car Age Category Analysis + Bar Chart

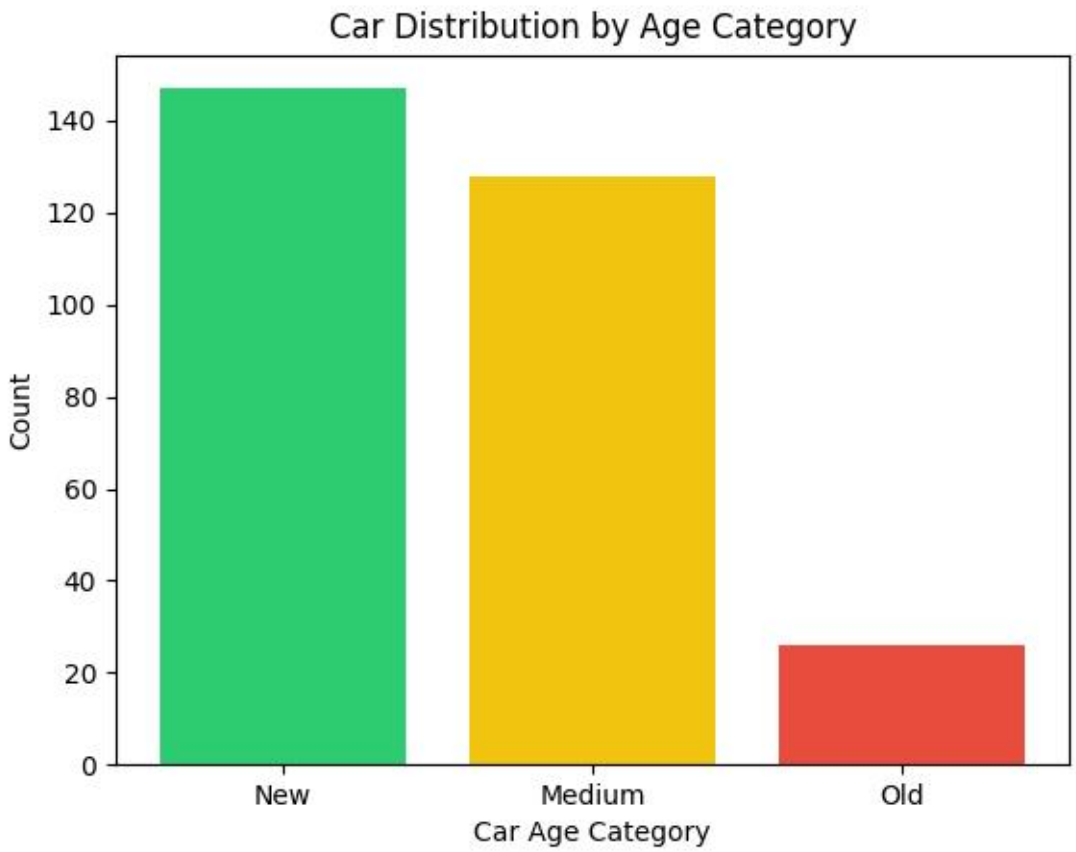
Description:

In this scenario, we creating a new column 'Car Age Category' based on the Year column to classify cars into New, Medium, and Old categories. We then count the number of cars in each category, convert the results into NumPy arrays, and plot a bar chart to compare the distribution of cars across age groups.

Task	Description
1.	Create a new column 'Car Age Category' using Pandas.
2.	Year >= 2015 → 'New', 2010 to 2014 → 'Medium', < 2010 → 'Old'.
3.	Count the number of cars in each Car Age Category.
4.	Convert category names and counts into NumPy arrays.
5.	Plot a bar chart: X-axis → Car Age Category, Y-axis → Count.
6.	Add title and labels.
7.	Save the graph.

Graph:

The bar chart displays the number of cars in each age category (New, Medium, Old) on the X-axis, with the count on the Y-axis. Each bar's height represents how many cars fall into that age bracket, giving a clear visual comparison of car age distribution in the dataset.



Conclusion:

The car age category analysis helps understand the age composition of cars in the dataset. The bar chart clearly shows which age group dominates the market.

Scenario 7: Kms Driven Distribution (Histogram)

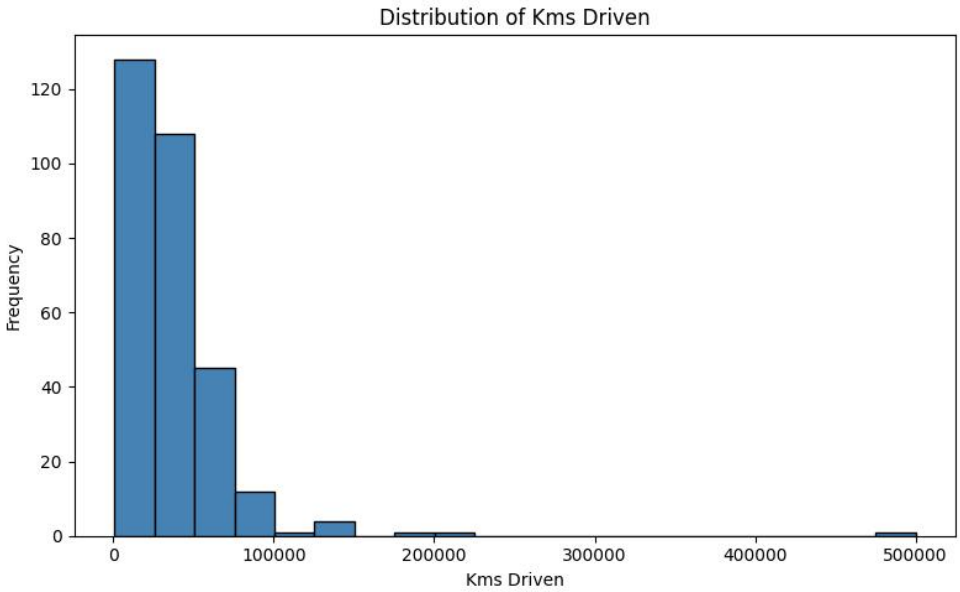
Description:

This scenario focuses on understanding how cars are distributed based on kilometers driven. We select the Kms_Driven column, convert it into a NumPy array, and plot a histogram with suitable bins to observe whether most cars have lower or higher mileage.

Task	Description
1.	Select the Kms_Driven column.
2.	Convert it into a NumPy array.
3.	Plot a histogram: X-axis → Kms Driven, Y-axis → Frequency.
4.	Choose a suitable number of bins.
5.	Add title, x-label, and y-label.
6.	Save the graph.
7.	Observe whether most cars have lower or higher mileage.

Graph:

The histogram displays the frequency distribution of Kms_Driven values in different ranges. It shows how many cars fall within each mileage interval. The shape of the distribution helps identify whether the dataset is dominated by low-mileage or high-mileage cars.



Conclusion:

The histogram reveals the mileage distribution of cars in the dataset. Most cars tend to be concentrated in the lower mileage ranges, indicating that the dataset contains a majority of relatively less-used cars. There are a few outliers with very high mileage. This insight is useful for understanding the overall usage patterns of vehicles in the dataset.

Scenario 8: Transmission-wise Selling Price Comparison

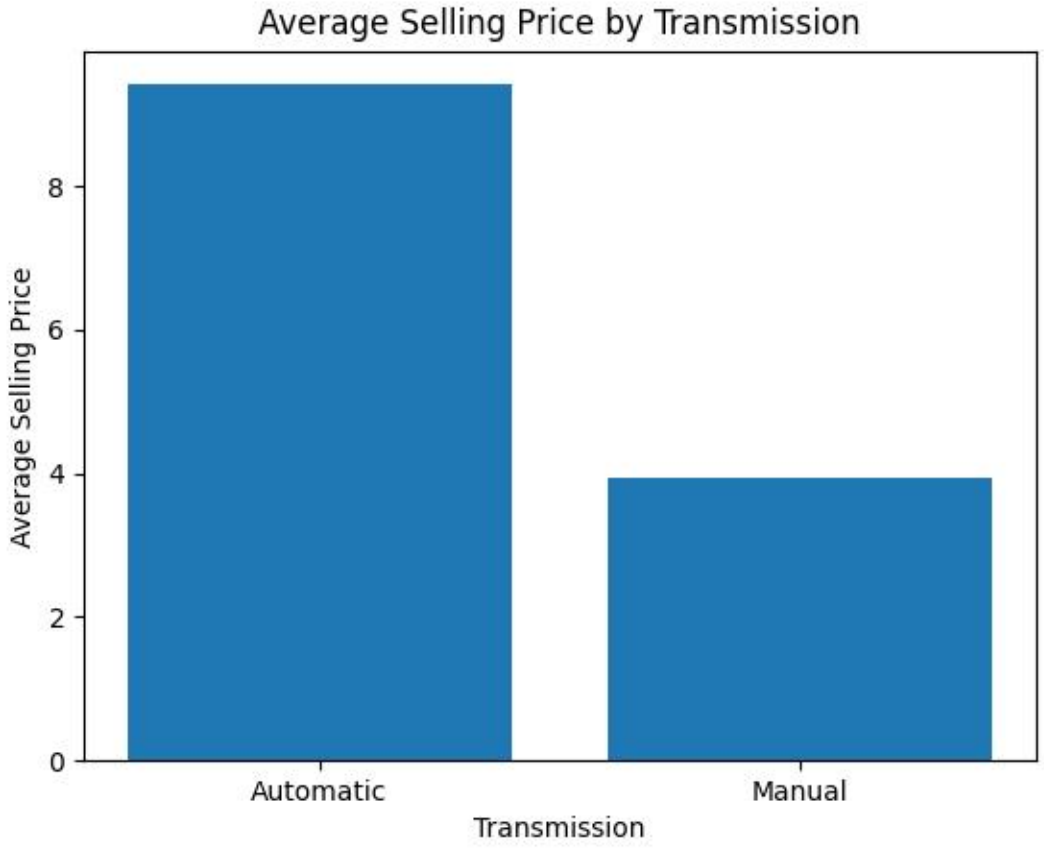
Description:

In this scenario, we analyze car data to compare the average selling prices of manual and automatic cars. The dataset is grouped based on the Transmission type, and the mean of Selling_Price is calculated for each group. This helps in understanding how pricing differs between transmission types.The transmission categories and their corresponding average prices are then converted into NumPy arrays for visualization. A bar chart is plotted where the X-axis represents transmission types (Manual, Automatic) and the Y-axis represents their average selling prices. Titles and labels are added for clarity, and the graph is saved as an image file.

Task	Description
1.	Group data by Transmission.
2.	Calculate average Selling_Price for each transmission type.
3.	Convert transmission labels and average prices into NumPy arrays.
4.	Plot a bar chart: X-axis → Transmission, Y-axis → Average Selling Price.
5.	Add title and labels.
6.	Save the graph.

Graph:

The bar chart displays two bars representing Manual and Automatic transmission types on the X-axis, with the average selling price on the Y-axis. The height of each bar directly reflects the average price for that transmission type, making it easy to compare the two groups.



Conclusion:

This analysis compares the average selling price of manual and automatic cars. By grouping the data based on transmission and calculating the mean price, we can clearly see which type tends to be more expensive. The bar chart makes this comparison easy to understand visually.

Scenario 9: Seller Type Analysis

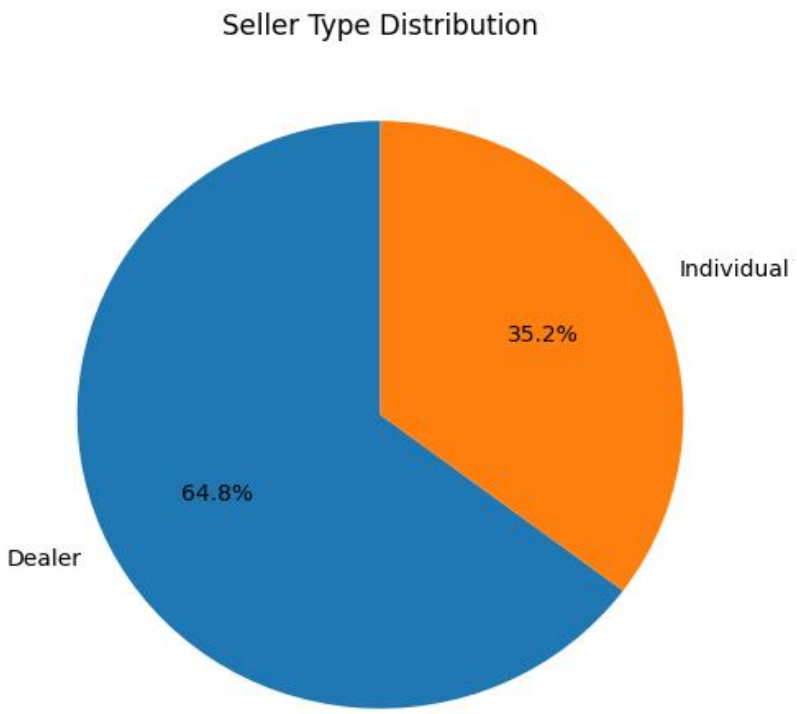
Description:

This scenario analyzes the distribution of cars sold by different seller types. It compares individual sellers and dealers in terms of total listings. The data is grouped and counted based on seller type. A pie chart is used for visualization.

Task	Description
1.	Count the number of cars by Seller_Type.
2.	Convert results into NumPy arrays.
3.	Plot a bar chart or pie chart using Matplotlib.
4.	Add labels and title.
5.	Save the graph.
6.	Identify which seller type is more common.

Graph:

The pie chart displays seller types as slices, with each slice representing its percentage share of total car sales. Labels such as Dealer and Individual are shown clearly. Percentage values are included on each slice for better understanding. The title highlights the comparison of seller types.



Conclusion:

The seller type with the largest slice is the most common in the dataset. This indicates whether dealers or individuals dominate car sales. Smaller slices represent less common seller types. The visualization provides a clear overview of seller distribution.

Scenario 10: Advanced Analysis + Multiple Graphs

Description:

This scenario introduces a new feature (Price Difference) to measure depreciation and uses NumPy for statistical insights. Multiple visualizations (line, bar, histogram) help analyze trends, fuel type impact, and price distribution. Results show how prices vary across categories and over entries.

Part 1: Feature Creation

Create a new column: Price Difference = Present_Price - Selling_Price. This shows how much value the car has depreciated.

Part 2: NumPy Usage

Task	Description
1.	Convert Selling_Price into a NumPy array.
2.	Use np.diff() to calculate price changes between consecutive rows.
3.	Convert Price Difference column into a NumPy array.
4.	Find average depreciation, maximum depreciation, and minimum depreciation.

Part 3: Visualizations

Task	Description
Line Graph	Plot Selling_Price trend for all cars.
Bar Chart	Show average Selling_Price by Fuel_Type.
Histogram	Plot distribution of Selling_Price.

Part 4: Insights

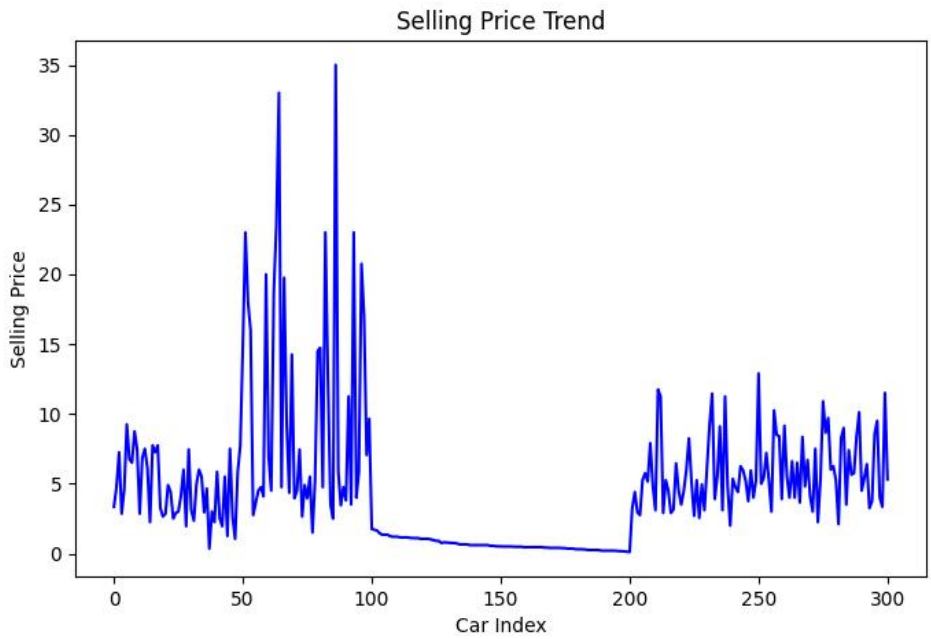
- Which fuel type has the highest average selling price?
- Which transmission type has higher average selling price?
- Are most cars concentrated in lower selling prices or higher selling prices?
- Do older cars tend to have lower selling prices?

Graphs:

Three graphs have been plotted — a Line graph, Bar Chart, and Histogram — for better visualization.

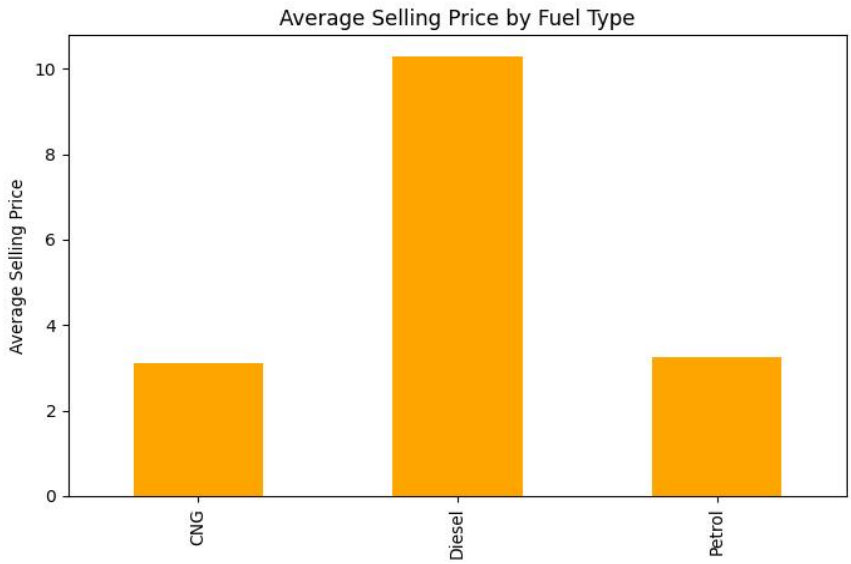
Line Graph

The line graph shows how selling prices vary across different cars in the dataset. It helps identify fluctuations and any visible trends over the sequence of entries. Peaks and dips indicate higher and lower priced cars.



Bar Chart

This bar chart shows the average selling price for each fuel type. Each bar represents a fuel type and its height shows the average selling price in that category. It helps compare how fuel type influences pricing.



Histogram

The histogram shows how selling prices are distributed across ranges. Most bars are concentrated on the lower price side, indicating more cars fall in that range. It reveals the spread and skewness of data.

Conclusion:

The analysis shows that cars generally lose value over time, confirming depreciation. Fuel type and transmission type both have a noticeable impact on average selling prices. The histogram indicates that most cars are concentrated in the lower price range. Variations in the line graph reflect differences in car features and conditions. Overall, pricing is influenced by multiple factors, with depreciation being a major one.

Final Conclusion

This project analysed the Car Data dataset using Pandas, NumPy, and Matplotlib. It showed that petrol cars dominate the dataset, while diesel cars tend to command higher selling prices. Most cars fall in the lower selling price range, indicating a right-skewed price distribution. Automatic transmission cars have a higher average selling price than manual ones. The data visualizations helped clearly understand patterns in car pricing, fuel type distribution, and mileage. Overall, the analysis gave useful insights into the used car market represented in the dataset.

What We Have Learned

In this project, we have learnt how to clean and process real-world car sales data using Pandas and perform numerical calculations using NumPy. We have also learnt how to create different visualizations like line graphs, bar charts, pie charts, scatter plots, and histograms using Matplotlib. Additionally, we learnt feature engineering techniques such as creating 'Car Age Category' and 'Price Difference' columns. Most importantly, we understood how to turn raw car sales data into meaningful insights using an end-to-end data analysis process.